# GUÍA DE LA ENTREGA 4 — Búsqueda, ordenamiento y complejidad

**Vence:** domingo **01-nov-2026, 23:59** (hora Argentina).
**Vale: 9 %** de la nota final.
**Tag en GitHub:** exactamente `entrega-4`.

> Esta guía tiene **teoría + práctica** para entender cada concepto desde cero.
> Léela entera antes de escribir una línea de código.

---

## 0. QUÉ SE CORRIGE ESTA VEZ (checklist rápido)

Son 6 ítems de la rúbrica. Si hacés solo esto, aprobás la E4:

| # | Qué se corrige | Peso interno | En palabras simples |
|---|---|---|---|
| 4.1 | Búsqueda **lineal** propia, usada desde el CLI | 15 % | Recorrer de a uno, sin `in` de Python ni `index()`. |
| 4.2 | Búsqueda **binaria** propia sobre colección **ya ordenada** | 20 % | Dividir el rango a la mitad. El menú no la ofrece si no está ordenado. |
| 4.3 | **Dos** ordenamientos propios, criterios distintos (o estabilidad distinta) | 25 % | Por selección y por inserción. **Prohibido** `sorted` / `list.sort`. |
| 4.4 | Tabla de complejidad de las operaciones del sistema | 20 % | En `docs/INFORME.md` §5: operación → cota → por qué. |
| 4.5 | Medición real con `time.perf_counter` | 15 % | Medir sobre el dataset original **y** una copia agrandada (o 2 tamaños). |
| 4.6 | Tag corre; no rompieron E2/E3 | 5 % | Regresión: recursión, TADs y listado intactos. |

**Regla de oro:** lo que andaba en E1, E2 y E3 tiene que seguir andando. Si rompés algo viejo, pierdes puntos del ítem nuevo aunque el nuevo esté perfecto.

---

## 1. DÓNDE VIVE EL CÓDIGO

Todo lo nuevo va en `src/algoritmos/` (ya está la carpeta en el esqueleto, vacía):

```
src/
├── main.py              # + opciones "Buscar" y "Ordenar"
├── dominio/             # intacto de E3
├── tads/                # intacto de E3
```

```
├── algoritmos/           # ← E4: acá vive todo
│   ├── busqueda.py       # busqueda_lineal, busqueda_binaria
│   └── ordenamiento.py   # ordenar_seleccion, ordenar_insercion
└── persistencia/
docs/
├── INFORME.md            # + §5 complejidad y mediciones
├── PROTOCOLO_PRUEBAS.md  # + casos P09 a P12 ejecutados
└── DECLARACION_IA.md     # + fila E4
```

No toques `data/*.txt` (se leen recién en E5) ni modifiques lo de E1–E3 salvo lo necesario para enchufar las nuevas opciones del menú.

---

# 2. TEORÍA: BÚSQUEDA

## 2.1 Búsqueda lineal (ítem 4.1 — 15 %)

**Qué es:** mirás el primer elemento, si no es el que buscás pasás al siguiente, y así hasta el final. Como buscar una persona en una fila mirando la cara de cada una.

**Ventaja:** funciona con la colección **en cualquier orden**.
**Desventaja:** si hay 1.000.000 de elementos, en el peor caso hacés 1.000.000 de comparaciones → **O(n)**.

```python
# src/algoritmos/busqueda.py

def busqueda_lineal(coleccion, clave, obtener_clave=None):
    """Devuelve el elemento cuya clave es igual a 'clave', o None si no existe.

    obtener_clave: función que de un elemento devuelve la clave a comparar
    (ej: lambda p: p.nombre). Si es None, compara el elemento entero.
    """
    for elemento in coleccion:
        if obtener_clave is None:
            if elemento == clave:
                return elemento
        elif obtener_clave(elemento) == clave:
            return elemento
    return None          # no encontrado: el CLI avisa, no explota
```

**¿Por qué `obtener_clave`?** Porque tu catálogo tiene objetos (`Pokemon`, `Receta`, `Cancion`) y a veces querés buscar por nombre, a veces por id, a veces por título. La función recibe la clave que hay que comparar y el algoritmo no se entera de qué busca.

## 2.2 Búsqueda binaria (ítem 4.2 — 20 %)

**Qué es:** solo funciona si la colección **ya está ordenada**. Mirás el elemento del **medio**: si el que buscás es más chico, descartás toda la mitad derecha; si es más grande, descartás la mitad izquierda. Y repetís sobre lo que queda.

```
[ 3, 7, 12, 18, 25, 31, 44 ]      buscamos 25
          ↑ medio = 18             25 > 18 → descartamos la mitad izquierda
[ 25, 31, 44 ]                     ↑ medio = 31   25 < 31 → descartamos la
derecha
[ 25 ]                             ↑ medio = 25   ¡lo encontramos!
```

**Ventaja:** cada paso corta el problema a la mitad → **O(log n)**. Con 1.000.000 de elementos, máximo ~20 comparaciones.
**Desventaja:** si la colección no está ordenada, devuelve resultados **falsos** (puede "no encontrar" algo que existe). Por eso la regla del CLI (más abajo).

```python
def busqueda_binaria(coleccion, clave, obtener_clave):
    """Búsqueda binaria. REQUIERE que la colección esté ordenada por esa
clave."""
    izq = 0
    der = len(coleccion) - 1
    while izq <= der:
        medio = (izq + der) // 2
        valor = obtener_clave(coleccion[medio])
        if valor == clave:
            return coleccion[medio]
        elif valor < clave:
            izq = medio + 1       # la clave está en la mitad derecha
        else:
            der = medio - 1       # la clave está en la mitad izquierda
    return None                   # recorrimos todo y no está
```

**Regla del menú (ítem 4.2):** la opción "buscar (binaria)" **solo se muestra / solo funciona si la colección está ordenada**. Dos caminos válidos:

1. No ofrecer la binaria hasta que el usuario ordene algo antes; o
2. Reordenar automáticamente y **avisar** con un mensaje ("Catálogo ordenado por nombre; usando búsqueda binaria").

Si la binaria corre sobre un catálogo desordenado y devuelve un falso "no encontrado", ese ítem es **N**.

> **Si tu catálogo vive en `ListaEnlazada` (E3):** la lista enlazada no tiene acceso aleatorio por índice. Opción simple: pasarlo a `list` para buscar (y anotar en el informe que esa conversión cuesta O(n)), o recorrer hasta la posición media. Lo que no se puede es hacer `lista[medio]` sobre una lista enlazada sin recorrer.

### 2.3 Prohibido

`bisect`, `bisect_left`, `bisect_right` o cualquier equivalente de la biblioteca estándar para cumplir este ítem. Usarlos = **N** en 4.2. (Pueden usarlos en un apéndice comparativo si quieren, pero el algoritmo propio tiene que estar.)

---

# 3. TEORÍA: ORDENAMIENTO

Tenés que escribir **dos** algoritmos de ordenamiento con la mano. La rúbrica exige **criterios distintos**, o **un criterio con estabilidad distinta**. Ejemplo válido: selección (por un criterio) + inserción (por otro criterio / estable).

## 3.1 Ordenamiento por selección (NO estable)

**Qué es:** recorrés la lista y, para cada posición, buscás el elemento más chico de todo lo que queda y lo dejás ahí. Como ordenar naipes: tomás el más chico de la mesa y lo dejás arriba a la izquierda, y seguís.

```python
# src/algoritmos/ordenamiento.py

def ordenar_seleccion(coleccion, obtener_clave):
    """Ordena la colección in place por selección (ascendente)."""
    n = len(coleccion)
    for i in range(n - 1):
        pos_min = i
        for j in range(i + 1, n):
            if obtener_clave(coleccion[j]) < obtener_clave(coleccion[pos_min]):
                pos_min = j
        coleccion[i], coleccion[pos_min] = coleccion[pos_min], coleccion[i]
    return coleccion
```

**Complejidad:** O($n^2$) en **todos** los casos (siempre recorre los dos bucles aunque ya esté ordenado).
**Estabilidad: no** es estable — al intercambiar elementos lejanos, dos elementos iguales pueden cambiar de orden relativo.

## 3.2 Ordenamiento por inserción (ESTABLE)

**Qué es:** como quien ordena naipes en la mano: tomás cada elemento y lo "insertás" en su lugar entre los ya ordenados, desplazando los que son más grandes.

```python
def ordenar_insercion(coleccion, obtener_clave):
    """Ordena la colección in place por inserción (ascendente)."""
    for i in range(1, len(coleccion)):
        actual = coleccion[i]
        j = i - 1
        while j >= 0 and obtener_clave(coleccion[j]) > obtener_clave(actual):
            coleccion[j + 1] = coleccion[j]     # desplazo el mayor a la
derecha
            j -= 1
        coleccion[j + 1] = actual               # inserto el elemento en su
lugar
    return coleccion
```

**Complejidad:** mejor caso **O(n)** (si ya está casi ordenado), peor y promedio **O($n^2$)**.
**Estabilidad: sí** es estable — dos elementos iguales conservan su orden original.

### 3.3 ¿Qué es la estabilidad? (para explicar la diferencia en el informe)

Dos elementos con la **misma clave** (ej: dos recetas de 30 minutos) deben quedar en el mismo orden relativo que tenían antes de ordenar. Selección **no** garantiza eso; inserción **sí**. Esa diferencia es un argumento válido para el ítem "criterios distintos o estable vs. no estable".

### 3.4 Prohibido

`sorted()`, `list.sort()`, `sorted(..., key=...)` para cumplir 4.3 = **N** en ese ítem. (Pueden mostrar un apéndice comparativo "mi algoritmo vs `sorted`" en el informe, pero el algoritmo propio tiene que existir y usarse desde el menú.)

### 3.5 Tercer algoritmo = nota extra

Si les sobra tiempo: **Shell sort** (o cualquier otro vistos en clase), medido en el informe → hasta **2 puntos extra** sobre las entregas (no pasan de 50).

---

## 4. CÓMO CONECTARLO AL CLI (ítems 4.1 y 4.2)

En `main.py`, las opciones 3 (Buscar) y 4 (Ordenar) del menú tienen que **usar** tus algoritmos:

```
from src.algoritmos.busqueda import busqueda_lineal, busqueda_binaria
from src.algoritmos.ordenamiento import ordenar_seleccion, ordenar_insercion
```

Ejemplo de flujo esperado:

```
=== Mi tema — AyED C2 2026 ===
3. Buscar
> 3
Buscar por: 1) Nombre (lineal)  2) Nombre (binaria)
> 1
Ingresá el nombre: Golem
Golem (Roca/Tierra, hp 80)

4. Ordenar
> 4
Ordenar por: 1) Nombre  2) Tiempo / hp / duración
> 1
[1] Seleccion... [2] Insercion
> 1
Catálogo ordenado por nombre (selección).
```

Checklist CLI:

- ☐ La búsqueda lineal se dispara desde una opción del menú.
- ☐ La binaria **solo** está disponible si la colección está ordenada (o reordena y avisa).
- ☐ Se puede ordenar por **dos criterios distintos** (ej: nombre y hp/tiempo/año) con cualquiera de los dos algoritmos.

---

# 5. ÍTEM 4.4 (20 %): TABLA DE COMPLEJIDAD EN EL INFORME

## 5.1 Qué es "complejidad" en una frase

Cuánto **crece** el tiempo de ejecución cuando crece el tamaño del problema, **sin contar la máquina**: se escribe con notación **O(...)** (orden grande).

## 5.2 Las cotas que tenés que usar

| Cota | Qué significa | Ejemplo |
|---|---|---|
| O(1) | Constante: tarda lo mismo con 10 o con 1.000.000 | Acceder a `lista[0]` de un `list` |
| O(log n) | Cada paso corta el problema a la mitad | Búsqueda binaria |
| O(n) | Crece lineal con los elementos | Búsqueda lineal, recorrer la lista enlazada |
| $O(n^2)$ | Un bucle dentro de otro | Selección, inserción (peor caso) |

## 5.3 Tabla a completar en `docs/INFORME.md` §5

| Operación | Tiempo | Espacio | Por qué |
|---|---|---|---|
| Listar catálogo | O(n) | O(1) | Recorre los n elementos una vez (iterador). |
| Búsqueda lineal | O(n) en el peor / O(1) en el mejor | O(1) | Toca cada elemento hasta encontrarlo o llegar al final. |
| Búsqueda binaria | O(log n) | O(1) | Cada comparación descarta la mitad del rango. |
| Ordenar por selección | $O(n^2)$ siempre | O(1) | Dos bucles anidados: $n + (n-1) + \ldots + 1 \approx n^2/2$ comparaciones. |
| Ordenar por inserción | O(n) mejor / $O(n^2)$ peor y promedio | O(1) | Si ya está ordenado, hace 1 comparación por elemento; si está al revés, desplaza todo. |
| Ver detalle de un ítem | O(n) o O(1) | O(1) | Según cómo accedan (recorrido vs. índice). |

**La justificación ("Por qué") es 1 o 2 frases, no un paper.** Lo que se pide es que se note que pensaron de dónde sale la cota.

## 6. ÍTEM 4.5 (15 %): MEDIR CON `time.perf_counter`

Lo que miden con `time.perf_counter()` va en la tabla de `docs/INFORME.md` §5:

| Operación | n | segundos |
|---|---|---|
| búsqueda lineal (nombre) | 71 | 0.000008 |
| búsqueda lineal (n × 10) | 710 | 0.000061 |
| búsqueda binaria (nombre) | 71 | 0.000002 |
| búsqueda binaria (n × 10) | 710 | 0.000003 |

**Cómo se mide:**

```
import time

inicio = time.perf_counter()
resultado = busqueda_lineal(datos, "Golem", obtener_clave=lambda p: p.nombre)
fin = time.perf_counter()
print(f"lineal: {fin - inicio:.6f} s")
```

**Cómo agrandar el dataset** (obligatorio: dataset original + una copia más grande, o al menos 2 tamaños):

```
# repetir las filas para simular un catálogo 10 veces más grande
datos_grandes = datos * 10
```

Lo que se busca que se note: la binaria **apenas** cambia al agrandar (log n), la lineal y el ordenamiento **se disparan** (n y $n^2$). No hace falta un paper: hace falta que se vea que midieron y que entendieron la tendencia.

---

## 7. PROTOCOLO DE PRUEBAS: P09 a P12 EJECUTADOS

En `docs/PROTOCOLO_PRUEBAS.md`, completá la columna **Resultado** de los casos P09–P12 con `pasa` o `no pasa` (y comento si algo salió raro):

| ID | Qué hacer en el CLI | Resultado esperado |
|---|---|---|
| P09 | Búsqueda lineal de un nombre que **existe** | lo encuentra y lo muestra |
| P10 | Búsqueda lineal de un nombre que **no existe** | "no encontrado", sin traceback, el menú sigue |
| P11 | Búsqueda binaria con el catálogo **desordenado** | avisa o reordena; **no** da un falso "no encontrado" |
| P12 | Ordenar por un criterio y después por **otro** | el orden cambia correctamente |

Si P11 sale mal, no hay que esconderlo: arreglar el programa (o la expectativa) y ahí queda `pasa`.

- [ ] Las 4 filas ejecutadas y completadas.
- [ ] Los casos de E1–E3 siguen en `pasa` (regresión).

---

## 8. DECLARACIÓN DE IA

En `docs/DECLARACION_IA.md`: completar la fila **E4** (fecha, herramienta, para qué, qué hicieron, quién). Si no usaron IA, igual pongan una fila que lo diga. **No declarar IA cuando sí se usó anula la entrega.**

---

## 9. TAG Y REGRESIÓN

```
git add .
git status                          # revisá que esté todo lo que querés subir
git commit -m "E4: búsqueda, orden y complejidad"
git tag entrega-4
git push
git push origin entrega-4
```

Verificá **antes** de tagear:

- [ ] `python -m src.main` arranca desde la raíz del repo.
- [ ] El listado del catálogo (E1) sigue funcionando.
- [ ] La recursión del dominio (E2) sigue funcionando.
- [ ] Lista enlazada, pila y cola (E3) siguen funcionando.

---

## 10. CÓMO RESPONDER / ENVIAR (obligatorio — léanlo dos veces)

**Paso 1 — tagear** (comandos de la sección 9). El tag tiene que ser exactamente `entrega-4`.

**Paso 2 — mandar el mail de la entrega** a los **dos** docentes en el **mismo** correo:

| Campo | Valor exacto |
|---|---|
| **Para** | `diego.ambrossio@unab.edu.ar; angel.bianco@unab.edu.ar` |
| **Asunto** | `AyED C2 2026 Comision X - GRUPO XX - Entrega 4` |
| **Cuerpo** | ver plantilla de abajo |

**Plantilla del cuerpo (copiar y pegar):**

```
Repo: https://github.com/<usuario>/<repo>
Entrega de E4: búsqueda lineal y binaria propias, dos ordenamientos
propios, tabla de complejidad y mediciones con time.perf_counter.
Tag: entrega-4.
```

**Ejemplo resuelto (así tiene que llegar):**

```
Para: diego.ambrossio@unab.edu.ar; angel.bianco@unab.edu.ar
Asunto: AyED C2 2026 Comision 1 - GRUPO 04 - Entrega 4

Repo: https://github.com/comision1-grupo4/ayed-2026-recetario
Entrega de E4: búsqueda lineal y binaria propias, dos ordenamientos
propios, tabla de complejidad y mediciones con time.perf_counter.
Tag: entrega-4.
```

**Paso 3 — campus:** cargar la URL del repo + el tag `entrega-4`.

> **Sin ese mail no corregimos la entrega** (igual que en E1, E2 y E3).

---

## 11. CHECKLIST FINAL ANTES DEL DOMINGO 23:59

- ☐ `src/algoritmos/busqueda.py` con `busqueda_lineal` y `busqueda_binaria` propias.
- ☐ `src/algoritmos/ordenamiento.py` con **dos** ordenamientos propios.
- ☐ Ningún uso de `sorted`, `list.sort` o `bisect` para cumplir los ítems.
- ☐ Menú: buscar (lineal) andando; binaria solo si está ordenado (o reordena y avisa).
- ☐ Menú: ordenar por dos criterios distintos.
- ☐ `docs/INFORME.md` §5 con la tabla de complejidad **y** las mediciones (original + agrandado).
- ☐ `docs/PROTOCOLO_PRUEBAS.md` con P09–P12 en `pasa` / `no pasa`.
- ☐ `docs/DECLARACION_IA.md` con la fila E4.
- ☐ E1, E2 y E3 siguen andando (listar, recursión, TADs).
- ☐ Tag `entrega-4` pusheado **y** `git push origin entrega-4`.
- ☐ Mail enviado a los dos docentes con el asunto y el cuerpo de la sección 10.

---

## 12. ERRORES QUE CUESTAN PUNTOS

| Error | Consecuencia según rúbrica | Cómo evitarlo |
|---|---|---|
| Usar `sorted` o `list.sort` para ordenar | Ítem 4.3 en **N** | Escribir selección e inserción a mano. |
| Usar `bisect` para la binaria | Ítem 4.2 en **N** | Copiar el algoritmo de la sección 2.2 y entenderlo. |
| Binaria sobre catálogo desordenado sin avisar | Ítem 4.2 en **N** | No ofrecer la opción, o reordenar y avisar con un mensaje. |
| Un solo criterio de ordenamiento | Ítem 4.3 en P o N | Dos criterios (ej: nombre y hp/tiempo/año). |

| Error | Consecuencia según rúbrica | Cómo evitarlo |
|---|---|---|
| Tabla de complejidad sin "por qué" | Ítem 4.4 en P | 1–2 frases de justificación por fila. |
| Una sola medición, sin dataset agrandado | Ítem 4.5 en P o N | Medir original ×1 y ×10 (o 2 tamaños distintos). |
| Sin casos P09–P12 en el protocolo | Descuenta en la corrección | Ejecutarlos y poner `pasa`/`no pasa`. |
| Roto algo de E1–E3 | Ítem 4.6 en N | Probar el menú completo antes de tagear. |
| Tag con otro nombre (`E4`, `entrega4`) | No se encuentra la entrega | Exactamente `entrega-4`. |
| Mail con otro asunto o sin los dos docentes | No se corrige | Copiar la plantilla de la sección 10. |

**Atraso:** hasta el martes 04-nov 23:59, con **−25 %** de esa entrega. Después no se acepta. Excepciones justificadas (salud, trámite) se avisan **antes** del vencimiento.

---

## 13. NOTA EXTRA (hasta 2 puntos sobre las entregas, no pasa de 50)

- Un **tercer** algoritmo de ordenamiento (ej: Shell), medido y comparado en el informe; **o**
- Usar la lista enlazada también en el catálogo **y** un índice `dict` como estructura auxiliar, comparados en el informe: **qué ganan** (búsqueda O(1)) **y qué pagan** (memoria, sincronización).

---

## 14. RECURSOS

- `entrega-4/entrega-4.md` — enunciado corto de la entrega con la rúbrica.
- `guia-github.md` — tutorial de Git/GitHub desde cero (clone, commit, push, tag).
- `tp-integrador/consigna.md` §3.6 y §3.7 — requisitos originales de búsqueda, ordenamiento y complejidad.
- `tp-integrador/rubrica.md` — checklist exacto con el que se corrige.

**¿Dudas?** En horario de consultas o por el campus. No dejes todo para el último día: la medición de tiempos siempre lleva más de lo que parece.